

Техническое задание

1. Наименование проекта

mdpdf

Автономная консольная утилита для преобразования Markdown-документов в PDF:

Markdown → собственный AST → Typst source → Typst compiler → PDF

Программа поставляется как один исполняемый файл.

2. Цель проекта

Разработать кроссплатформенную CLI-утилиту на Rust, которая:

1. читает Markdown из файла или стандартного ввода;
 2. преобразует Markdown в собственное типизированное AST;
 3. генерирует валидный Typst-документ;
 4. компилирует Typst-документ во встроенном Typst-компиляторе;
 5. записывает готовый PDF;
 6. не требует установки Typst, Chromium, LaTeX, Pandoc или иных внешних программ;
 7. не использует сеть;
 8. не загружает Typst-пакеты;
 9. поставляется одним бинарным файлом.
-

3. Основные ограничения

3.1. Обязательные ограничения

- Язык реализации: Rust.
- Rust edition: 2024.
- Основной Markdown-парсер: pulldown-cmark.
- Typst должен использоваться как Rust-библиотека.
- PDF должен формироваться библиотекой typst-pdf.
- Вызов внешнего typst CLI запрещён.
- Вызов внешних процессов запрещён.
- Использование Chromium запрещено.
- Использование WebView запрещено.
- Использование HTML/CSS как промежуточного формата запрещено.
- Использование Pandoc запрещено.
- Использование LaTeX запрещено.
- Сетевой доступ запрещён.
- Загрузка ресурсов по HTTP/HTTPS запрещена.

- Typst Universe и любые внешние Typst-пакеты запрещены.
- Основные шрифты должны быть встроены в бинарный файл.
- Шаблон оформления должен быть встроен в бинарный файл.
- Временные файлы для нормальной работы использоваться не должны.
- Парсинг Markdown не должен быть связан с Typst.
- AST не должен содержать Typst-синтаксис.
- Модуль компиляции Typst не должен знать о Markdown.
- Ошибки не должны приводить к `panic!`.
- В production-коде запрещены `unwrap()` и `expect()`.

3.2. Не входит в первую версию

- HTML внутри Markdown;
- выполнение JavaScript;
- Mermaid;
- PlantUML;
- математический LaTeX;
- библиография;
- сноски;
- пользовательские Typst-шаблоны;
- пользовательские Typst-скрипты;
- внешние Typst-пакеты;
- сетевые изображения;
- SVG с внешними ссылками;
- PDF/A;
- PDF/UA;
- цифровая подпись PDF;
- шифрование PDF;
- интерактивные PDF-формы;
- GUI;
- серверный режим;
- режим наблюдения за файлами;
- второй backend.

4. Поддерживаемые платформы

Минимально:

- macOS ARM64;
- macOS x86-64;
- Linux x86-64;
- Linux ARM64;
- Windows x86-64.

Для каждой платформы собирается отдельный автономный бинарный файл.

Абсолютная переносимость одного и того же бинарника между операционными системами не требуется.

5. Пользовательский интерфейс

5.1. Основная команда

```
mdpdf input.md
```

При отсутствии `--output` результат записывается рядом с исходным файлом:

```
input.md → input.pdf
```

5.2. Явный выходной файл

```
mdpdf input.md --output output.pdf
```

Краткая форма:

```
mdpdf input.md -o output.pdf
```

5.3. Чтение из stdin

```
cat input.md | mdpdf - --output output.pdf
```

При входе из stdin параметр `--output` обязателен.

5.4. Проверка без создания PDF

```
mdpdf input.md --check
```

Команда выполняет:

1. чтение Markdown;
2. парсинг Markdown;
3. построение AST;
4. валидацию AST;
5. генерацию Typst;
6. компиляцию Typst.

PDF не записывается.

5.5. Вывод сгенерированного Typst

```
mdpdf input.md --emit-typst output.typ
```

Этот режим предназначен для диагностики и тестирования.

При его использовании:

- AST строится;
- Typst генерируется;
- результат записывается в указанный файл;
- PDF не создаётся, если одновременно не передан `--output`.

5.6. Диагностический вывод AST

```
mdpdf input.md --emit-ast output.json
```

AST сериализуется в JSON.

Опция доступна только в диагностических целях. JSON не является стабильным публичным форматом первой версии.

5.7. Остальные параметры

```
mdpdf [OPTIONS] <INPUT>
```

Arguments:

<INPUT> Markdown-файл или "-" для stdin

Options:

<code>-o, --output <FILE></code>	Выходной PDF
<code>--title <TEXT></code>	Переопределить заголовок документа
<code>--author <TEXT></code>	Указать автора
<code>--paper <PAPER></code>	a4 или letter [default: a4]
<code>--margin <LENGTH></code>	Поля страницы [default: 20mm]
<code>--font-size <LENGTH></code>	Основной размер текста [default: 11pt]
<code>--toc</code>	Создать оглавление
<code>--heading-numbers</code>	Нумеровать заголовки
<code>--check</code>	Проверить документ без записи PDF
<code>--emit-ast <FILE></code>	Записать AST в JSON
<code>--emit-typst <FILE></code>	Записать сгенерированный Typst
<code>--overwrite</code>	Разрешить замену выходного файла
<code>--quiet</code>	Не выводить сообщения об успехе
<code>--verbose</code>	Расширенная диагностика
<code>-h, --help</code>	Справка
<code>-V, --version</code>	Версия

6. Поведение файловой системы

6.1. Разрешённые операции

Программа может:

- прочитать входной Markdown;
- прочитать локальные изображения, указанные в Markdown;
- создать выходной PDF;
- создать диагностические AST или Typst-файлы;
- читать метаданные локальных файлов.

6.2. Запрещённые операции

Программа не должна:

- обращаться к сети;
- создавать кеш в домашнем каталоге;
- устанавливать пакеты;
- запускать процессы;
- читать произвольные файлы, не связанные с документом;
- изменять входные файлы;
- молча перезаписывать существующий PDF без `--overwrite`.

6.3. Относительные пути

Путь изображения:

```
![Схема](images/schema.png)
```

разрешается относительно каталога Markdown-файла.

При чтении из stdin относительные пути разрешаются относительно текущего рабочего каталога.

6.4. Атомарная запись результата

PDF должен записываться атомарно:

1. PDF полностью формируется в памяти;
2. создаётся временный файл в каталоге назначения;
3. выполняется запись;
4. выполняется `flush`;
5. временный файл переименовывается в конечный.

При ошибке существующий выходной файл не должен повреждаться.

7. Структура проекта

```
mdpdf/
├── AGENTS.md
├── Cargo.toml
├── Cargo.lock
├── LICENSE
├── README.md
├── deny.toml
├── rustfmt.toml
├── assets/
│   ├── template.typ
│   └── fonts/
│       ├── NotoSans-Regular.ttf
│       ├── NotoSans-Bold.ttf
│       ├── NotoSans-Italic.ttf
│       ├── NotoSans-BoldItalic.ttf
│       └── NotoSansMono-Regular.ttf
├── src/
│   ├── main.rs
│   ├── cli.rs
│   ├── app.rs
│   ├── config.rs
│   ├── error.rs
│   ├── source.rs
│   ├── ast/
│   │   ├── mod.rs
│   │   ├── document.rs
│   │   ├── block.rs
│   │   ├── inline.rs
│   │   ├── metadata.rs
│   │   └── validate.rs
│   ├── markdown/
│   │   ├── mod.rs
│   │   ├── parser.rs
│   │   ├── builder.rs
│   │   ├── state.rs
│   │   └── error.rs
│   ├── typst_gen/
│   │   ├── mod.rs
│   │   ├── generator.rs
│   │   ├── writer.rs
│   │   ├── escape.rs
│   │   ├── blocks.rs
│   │   ├── inlines.rs
│   │   └── error.rs
│   ├── compiler/
│   │   ├── mod.rs
│   │   └── world.rs
```

```

|   |   |— fonts.rs
|   |   |— files.rs
|   |   |— diagnostics.rs
|   |   |— pdf.rs
|   |   |— error.rs
|   |— output.rs
|— tests/
|   |— fixtures/
|   |   |— markdown/
|   |   |— images/
|   |   |— expected_ast/
|   |   |— expected_typst/
|   |— markdown_parser.rs
|   |— typst_generator.rs
|   |— compiler.rs
|   |— cli.rs
|   |— end_to_end.rs

```

Этап 1. Парсинг Markdown в AST

8. Назначение этапа

Реализовать независимое преобразование:

`&str Markdown` → `Document AST`

Этап не должен:

- генерировать Typst;
- знать синтаксис Typst;
- компилировать PDF;
- читать шрифты;
- работать с PDF;
- выполнять файловый вывод.

9. Зависимость Markdown-парсера

Использовать `pulldown-cmark`.

Включить расширения:

- tables;
- task lists;
- strikethrough;

- heading attributes — только если версия парсера поддерживает их без собственной грамматики;
- metadata blocks — только после отдельного решения.

Не включать:

- footnotes;
- raw HTML rendering;
- smart punctuation, если оно необратимо изменяет исходный текст;
- неизвестные нестандартные расширения.

Конкретный набор `Options` должен находиться в одном месте:

```
fn markdown_options() -> pulldown_cmark::Options;
```

10. Требования к AST

10.1. Основной документ

```
#[derive(Debug, Clone, PartialEq)]
pub struct Document {
    pub metadata: DocumentMetadata,
    pub blocks: Vec<Block>,
}
```

10.2. Метаданные

```
#[derive(Debug, Clone, Default, PartialEq)]
pub struct DocumentMetadata {
    pub title: Option<String>,
    pub author: Option<String>,
    pub language: Option<String>,
}
```

В первой версии метаданные поступают из CLI.

Извлечение YAML front matter не реализуется.

10.3. Блочные элементы

```
#[derive(Debug, Clone, PartialEq)]
pub enum Block {
    Heading(Heading),
    Paragraph(Paragraph),
    CodeBlock(CodeBlock),
}
```

```

    Quote(BlockQuote),
    List(List),
    Table(Table),
    ThematicBreak,
}

```

10.4. Заголовок

```

#[derive(Debug, Clone, PartialEq)]
pub struct Heading {
    pub level: HeadingLevel,
    pub content: Vec<Inline>,
    pub id: Option<String>,
}

```

```

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum HeadingLevel {
    H1,
    H2,
    H3,
    H4,
    H5,
    H6,
}

```

Недопустимые уровни заголовков не должны представляться обычным `u8`.

10.5. Абзац

```

#[derive(Debug, Clone, PartialEq)]
pub struct Paragraph {
    pub content: Vec<Inline>,
}

```

10.6. Кодовый блок

```

#[derive(Debug, Clone, PartialEq)]
pub struct CodeBlock {
    pub language: Option<String>,
    pub code: String,
}

```

Требования:

- содержимое сохраняется без интерпретации;
- завершающий перевод строки нормализуется единообразно;

- язык очищается от пробелов;
- дополнительные параметры после языка в первой версии игнорируются либо возвращают диагностическое предупреждение;
- неизвестный язык допустим.

10.7. Цитата

```
#[derive(Debug, Clone, PartialEq)]
pub struct BlockQuote {
    pub blocks: Vec<Block>,
}
```

Цитаты могут быть вложенными.

10.8. Список

```
#[derive(Debug, Clone, PartialEq)]
pub struct List {
    pub kind: ListKind,
    pub items: Vec<ListItem>,
}
```

```
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum ListKind {
    Unordered,
    Ordered { start: u64 },
}
```

```
#[derive(Debug, Clone, PartialEq)]
pub struct ListItem {
    pub checked: Option<bool>,
    pub blocks: Vec<Block>,
}
```

Требования:

- поддерживать вложенные списки;
- сохранять начальный номер нумерованного списка;
- поддерживать task-list items;
- не хранить маркер `-`, `*` или `+`, поскольку он не влияет на семантику;
- не хранить конкретный вид нумерации.

10.9. Таблица

```
#[derive(Debug, Clone, PartialEq)]
pub struct Table {
```

```

    pub alignments: Vec<Alignment>,
    pub header: TableRow,
    pub rows: Vec<TableRow>,
}

```

```

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum Alignment {
    None,
    Left,
    Center,
    Right,
}

```

```

#[derive(Debug, Clone, PartialEq)]
pub struct TableRow {
    pub cells: Vec<TableCell>,
}

```

```

#[derive(Debug, Clone, PartialEq)]
pub struct TableCell {
    pub content: Vec<Inline>,
}

```

В первой версии ячейка содержит только inline-элементы.

Количество ячеек каждой строки после нормализации должно совпадать с количеством столбцов:

- отсутствующие ячейки дополняются пустыми;
- лишние ячейки являются ошибкой AST validation.

10.10. Inline-элементы

```

#[derive(Debug, Clone, PartialEq)]
pub enum Inline {
    Text(String),
    SoftBreak,
    HardBreak,
    Emphasis(Vec<Inline>),
    Strong(Vec<Inline>),
    Strikethrough(Vec<Inline>),
    Code(String),
    Link(Link),
    Image(Image),
}

```

10.11. Ссылка

```
#[derive(Debug, Clone, PartialEq)]
pub struct Link {
    pub destination: String,
    pub title: Option<String>,
    pub content: Vec<Inline>,
}
```

Требования:

- `http`, `https`, `mailto` допускаются как адреса ссылки;
- ссылка не загружается;
- относительные ссылки сохраняются;
- пустой `destination` допустим;
- содержимое ссылки может включать форматирование;
- вложенная ссылка является ошибкой валидации.

10.12. Изображение

```
#[derive(Debug, Clone, PartialEq)]
pub struct Image {
    pub source: String,
    pub title: Option<String>,
    pub alt: Vec<Inline>,
}
```

Требования:

- `source` сохраняется на этапе парсинга без чтения файла;
- чтение файла выполняется только на этапе компиляции;
- URL с `http` или `https` должен быть отклонён до компиляции;
- `data:` URI не поддерживается;
- `alt`-текст хранится структурированно;
- вложенные изображения запрещены.

11. Исходные позиции

Каждый узел верхнего уровня должен иметь диагностический диапазон исходного Markdown:

```
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub struct SourceSpan {
    pub start: usize,
    pub end: usize,
}
```

Для этого допускается обёртка:

```
pub struct Spanned<T> {  
    pub value: T,  
    pub span: SourceSpan,  
}
```

Минимально spans обязательны для:

- заголовков;
- абзацев;
- блоков кода;
- списков;
- таблиц;
- изображений;
- ссылок.

Диапазоны задаются в байтах UTF-8 и должны соответствовать диапазонам исходной строки.

Преобразование в номера строк и столбцов выполняется только при построении диагностического сообщения.

12. Алгоритм построения AST

`pull-down-mark` выдаёт поток событий. Необходимо реализовать конечный автомат со стеком контейнеров.

12.1. Контекст построения

```
struct AstBuilder {  
    document: Document,  
    stack: Vec<Frame>,  
}
```

12.2. Кадры стека

```
enum Frame {  
    Paragraph(InlineFrame),  
    Heading(HeadingFrame),  
    Emphasis(InlineFrame),  
    Strong(InlineFrame),  
    Strikethrough(InlineFrame),  
    Link(LinkFrame),  
    Image(ImageFrame),  
    CodeBlock(CodeBlockFrame),  
    Quote(BlockFrame),  
}
```

```

List(ListFrame),
ListItem(BlockFrame),
Table(TableFrame),
TableHead(TableSectionFrame),
TableRow(TableRowFrame),
TableCell(InlineFrame),
}

```

Не допускается единый кадр с большим количеством `Option`-полей.

12.3. Инварианты builder

- `Start(Tag)` создаёт соответствующий frame.
- `End(TagEnd)` обязан закрыть frame совместимого типа.
- Несовместимое закрытие является внутренней ошибкой парсера.
- Текст добавляется только в контейнер, допускающий inline.
- Блочный элемент добавляется только в блочный контейнер.
- После завершения потока stack должен быть пуст.
- Нельзя молча терять события.
- Каждое неподдерживаемое событие должно:
 - либо возвращать понятную ошибку;
 - либо создавать явно документированное предупреждение.

13. Нормализация текста

На этапе чтения:

- UTF-8 BOM в начале файла удаляется;
- `CRLF` преобразуется в `LF`;
- одиночный `CR` преобразуется в `LF`;
- остальные Unicode-символы сохраняются;
- Unicode normalization NFC/NFKC не выполняется;
- trailing spaces сохраняются до обработки Markdown-парсером;
- нулевой байт является ошибкой входа.

14. Валидация AST

После построения выполняется отдельная функция:

```

pub fn validate_document(document: &Document) -> Result<(),
AstValidationError>;

```

Проверяются:

- пустой стек уже исключён builder;
- уровень заголовка допустим;

- заголовок не содержит блочных узлов;
- таблица содержит хотя бы один столбец;
- число alignments совпадает с числом столбцов;
- строки таблицы имеют одинаковое число ячеек;
- список содержит хотя бы один item;
- список не содержит номера `0`;
- вложенные ссылки отсутствуют;
- вложенные изображения отсутствуют;
- URL изображения не является сетевым;
- `SourceSpan.start <= SourceSpan.end`;
- span не выходит за пределы исходного документа;
- кодовый блок содержит допустимый UTF-8;
- AST не содержит заведомо невозможных пустых служебных контейнеров.

Пустой Markdown является корректным документом.

15. API этапа 1

```
pub struct MarkdownParser {
    options: MarkdownOptions,
}

impl MarkdownParser {
    pub fn new(options: MarkdownOptions) -> Self;

    pub fn parse(
        &self,
        source: &str,
    ) -> Result<Document, MarkdownError>;
}
```

`MarkdownOptions` является собственной структурой проекта и не экспортирует типы `pull-down-smark` наружу.

16. Ошибки этапа 1

```
pub enum MarkdownError {
    InvalidInput {
        message: String,
        span: Option<SourceSpan>,
    },
    UnsupportedConstruct {
        construct: String,
        span: SourceSpan,
    },
    InvalidNesting {
```

```

        expected: String,
        actual: String,
        span: SourceSpan,
    },
    IncompleteDocument {
        open_construct: String,
        span: SourceSpan,
    },
    AstValidation(AstValidationError),
    InternalInvariant {
        message: String,
    },
}

```

Сообщение пользователю должно содержать:

```
input.md:12:5: unsupported Markdown construct: inline HTML
```

В verbose-режиме допускается дополнительный внутренний контекст.

17. Критерии готовности этапа 1

Этап считается завершённым, когда:

- Markdown преобразуется в собственный AST;
- AST не зависит от Typst;
- поддерживаются все элементы первой версии;
- все узлы проходят validation;
- есть unit-тесты каждого типа узла;
- есть тесты вложенных списков;
- есть тесты вложенных цитат;
- есть тесты таблиц;
- есть тесты Unicode и кириллицы;
- есть тесты неправильного вложения;
- есть snapshot/golden-тесты AST;
- покрытие модуля `markdown` и `ast` составляет не менее 90% строк;
- `cargo clippy --all-targets --all-features -- -D warnings` проходит;
- fuzz-тест парсера не вызывает panic.

Этап 2. Генератор Typst

18. Назначение этапа

Реализовать чистое преобразование:

Document AST + RenderOptions → String с Typst source

Генератор:

- не читает файлы;
- не компилирует Typst;
- не создаёт PDF;
- не обращается к сети;
- не зависит от `pulldown-smark`;
- не изменяет AST.

19. API генератора

```
pub struct TypstGenerator {
    options: RenderOptions,
}

impl TypstGenerator {
    pub fn new(options: RenderOptions) -> Self;

    pub fn generate(
        &self,
        document: &Document,
    ) -> Result<GeneratedTypst, TypstGenerationError>;
}
```

```
pub struct GeneratedTypst {
    pub source: String,
    pub resources: Vec<ResourceReference>,
}
```

```
pub struct ResourceReference {
    pub logical_path: String,
    pub kind: ResourceKind,
    pub span: Option<SourceSpan>,
}
```

```
pub enum ResourceKind {
    Image,
}
```

`resources` требуется, чтобы compiler заранее знал, какие локальные ресурсы разрешены.

20. RenderOptions

```
pub struct RenderOptions {  
    pub paper: PaperSize,  
    pub margin: Length,  
    pub font_size: Length,  
    pub title: Option<String>,  
    pub author: Option<String>,  
    pub toc: bool,  
    pub heading_numbers: bool,  
}
```

20.1. PaperSize

```
pub enum PaperSize {  
    A4,  
    Letter,  
}
```

20.2. Length

Строковые произвольные Turst-выражения запрещены.

Использовать типизированную модель:

```
pub struct Length {  
    pub value: f64,  
    pub unit: LengthUnit,  
}  
  
pub enum LengthUnit {  
    Pt,  
    Mm,  
    Cm,  
    In,  
}
```

Проверки:

- значение конечное;
- значение положительное;
- margin не более половины меньшей стороны страницы;
- font size находится в диапазоне `6pt..=72pt`.

21. Встроенный шаблон

Базовый Typst-шаблон встраивается через `include_str!`.

Шаблон отвечает только за:

- размер страницы;
- поля;
- основной шрифт;
- моноширинный шрифт;
- размер текста;
- межстрочный интервал;
- оформление заголовков;
- оформление цитат;
- оформление кода;
- оформление таблиц;
- номера страниц;
- оглавление;
- нумерацию заголовков.

Шаблон не должен:

- импортировать пакеты;
- читать сеть;
- читать произвольные файлы;
- выполнять нестабильные вычисления;
- зависеть от текущего времени;
- зависеть от окружения;
- использовать случайные значения.

Пример композиции:

```
<встроенный preamble>
<метаданные>
<настройки документа>
<сгенерированное тело>
```

22. Правило генерации Typst

Предпочтение отдаётся функциональной форме Typst, а не незкранированной markup-разметке.

Например, пользовательский текст не должен вставляться так:

```
Пользовательский #текст [как есть]
```

Он должен передаваться как безопасно сформированный строковый литерал или content-конструкция.

Основной принцип:

Ни один пользовательский фрагмент не должен интерпретироваться как Typst-код.

Это относится к:

- обычному тексту;
- заголовкам;
- alt-тексту;
- URL;
- title ссылок;
- имени языка кода;
- метаданным;
- автору;
- названию документа;
- путям изображений.

23. Экранирование

Реализовать отдельный модуль:

```
pub fn string_literal(value: &str) -> String;
pub fn text_content(value: &str) -> String;
pub fn url_literal(value: &str) -> Result<String, EscapeError>;
pub fn path_literal(value: &str) -> Result<String, EscapeError>;
```

Нельзя использовать одну универсальную функцию для всех контекстов.

23.1. String literal

Обязательно экранировать:

- `\`;
- `"`;
- переводы строк;
- carriage return;
- tab;
- управляющие Unicode-символы;
- нулевой байт.

23.2. Text content

Должно исключаться выполнение:

- `#`;
- `[` и `]` в контексте content;
- `$` в math-контексте;
- обратной косой черты;

- комментариев Typst;
- raw delimiters.

23.3. Кодовый блок

Код нельзя вставлять в raw-блок с фиксированным количеством обратных кавычек без проверки содержимого.

Разрешённые решения:

1. безопасный Typst string/bytes API;
2. подбор raw-delimiter, отсутствующего в содержимом;
3. собственная функция шаблона, принимающая строку.

Предпочтительный вариант — передача кода как строкового значения в функцию шаблона.

24. Соответствие AST → Typst

24.1. Document

Генерируется:

1. preamble;
2. metadata;
3. optional title block;
4. optional outline;
5. body.

24.2. Heading

Соответствие:

```
H1 → level 1
H2 → level 2
...
H6 → level 6
```

Содержимое заголовка передаётся как content.

При включённом `heading_numbers` нумерация выполняется шаблоном Typst, а не Rust-кодом.

24.3. Paragraph

Inline-элементы объединяются в один Typst content block.

Абзац должен сохранять:

- soft break как пробельный перенос;

- hard break как принудительный перевод строки.

24.4. Emphasis

Emphasis	→ Typst emph
Strong	→ Typst strong
Strikethrough	→ Typst strike
Inline code	→ функция шаблона inline-code

24.5. Link

Ссылка генерируется как Typst link.

Текст ссылки может быть форматированным.

URL передаётся только как строковое значение.

Сетевой URL допустим как ссылка, поскольку он не загружается.

24.6. Image

Изображение генерируется через функцию шаблона.

В Typst source передаётся логический URI, а не произвольный абсолютный путь.

Пример внутреннего URI:

```
/mdpdpf-resources/000001.png
```

Соответствие URI реальному файлу хранится в `GeneratedTypst.resources` и предоставляется compiler world.

Это исключает прямой доступ Typst к файловой системе.

24.7. CodeBlock

Генерируется через функцию:

```
#mdpdpf-code(language: "...", body: "...")
```

Требования:

- сохранить пробелы;
- сохранить переводы строк;
- использовать встроенный моноширинный шрифт;
- разрешить перенос слишком длинных строк либо горизонтальное уменьшение в пределах блока;

- не выполнять syntax highlighting в первой версии;
- язык выводить как небольшую подпись, если он задан.

24.8. Quote

Генерируется:

```
#mdpdf-quote[
  ...
]
```

Поддерживается вложенное содержимое.

24.9. List

Маркированный список:

```
#mdpdf-list(
  ordered: false,
  items: (...)
)
```

Нумерованный список:

```
#mdpdf-list(
  ordered: true,
  start: N,
  items: (...)
)
```

Task-list item должен содержать отдельное логическое состояние:

```
checked = true
checked = false
checked = none
```

Не следует генерировать символы Unicode ☐ и ☒ непосредственно в Rust-коде; отображение определяется шаблоном.

24.10. Table

Генерируется типизированный вызов функции шаблона с:

- числом столбцов;
- alignments;
- header cells;

- body rows.

Не вставлять таблицу через ручную строковую Typst-разметку.

24.11. ThematicBreak

Генерируется вызов встроенной функции шаблона:

```
#mdpdf-rule()
```

25. Детерминированность генерации

При одинаковых:

- AST;
- RenderOptions;
- версии программы;

генератор должен выдавать побайтно одинаковый Typst source.

Запрещены:

- timestamp;
- случайные идентификаторы;
- hash map iteration без сортировки;
- абсолютные локальные пути;
- сведения об операционной системе;
- текущий рабочий каталог.

Логические идентификаторы ресурсов назначаются последовательно в порядке обхода AST:

```
/mdpdf-resources/000001  
/mdpdf-resources/000002  
...
```

26. Форматирование сгенерированного Typst

Генерируемый source должен быть читаемым:

- отступ: 2 пробела;
- перевод строки: LF;
- одна пустая строка между верхнеуровневыми блоками;
- стабильный порядок параметров;
- отсутствие trailing whitespace;
- завершающий LF обязателен.

Генератор не обязан запускать внешний formatter.

27. Ошибки этапа 2

```
pub enum TypstGenerationError {
  InvalidDocument {
    message: String,
    span: Option<SourceSpan>,
  },
  UnsupportedNode {
    node: String,
    span: Option<SourceSpan>,
  },
  InvalidUrl {
    value: String,
    span: Option<SourceSpan>,
  },
  InvalidImagePath {
    value: String,
    span: Option<SourceSpan>,
  },
  InvalidOption {
    name: String,
    message: String,
  },
  Escaping {
    context: String,
    message: String,
  },
  InternalInvariant {
    message: String,
  },
}
```

28. Безопасность генератора

Обязательные тестовые строки:

```
#panic("injected")
#include "secret"
#import "@preview/package"
$ x + y $
[content]
"quoted"
\escaped
```

```
// comment
/* comment */
```

Все они должны отображаться как обычный пользовательский текст и не исполняться.

Отдельно тестировать injection через:

- title;
- author;
- heading;
- paragraph;
- URL;
- image path;
- code language;
- alt-text;
- table cell;
- list item.

29. Критерии готовности этапа 2

Этап считается завершённым, когда:

- каждый AST-узел преобразуется в Typst;
- generator не зависит от Markdown parser;
- generator не читает файловую систему;
- generator не вызывает compiler;
- пользовательский ввод не исполняется как Typst-код;
- output детерминирован;
- golden-тесты Typst проходят;
- есть injection-тесты;
- есть тесты кириллицы;
- есть тесты вложенных структур;
- есть тесты таблиц;
- есть тесты code blocks с кавычками и backticks;
- `cargo clippy` проходит без предупреждений;
- покрытие `typst_gen` не менее 90% строк.

Этап 3. Встроенная компиляция Typst → PDF

30. Назначение этапа

Реализовать преобразование:

```
GeneratedTypst + локальные разрешённые ресурсы → Vec8 PDF
```

Этап использует встроенные библиотеки Typst.

Внешний Typst CLI не допускается.

31. Изоляция Typst API

Вся зависимость от внутренних API Typst должна находиться в:

```
src/compiler/
```

Остальные модули проекта не должны импортировать:

- `typst::*`;
- `typst_pdf::*`;
- типы Typst diagnostics;
- Typst file IDs;
- Typst world types.

Публичный интерфейс проекта:

```
pub trait PdfCompiler {  
    fn compile(  
        &self,  
        input: CompileInput<'_>,  
    ) -> Result<Vec<u8>, CompileError>;  
}
```

```
pub struct CompileInput<'a> {  
    pub typst_source: &'a str,  
    pub source_name: &'a str,  
    pub base_dir: &'a Path,  
    pub resources: &'a [ResourceReference],  
}
```

Реализация:

```
pub struct EmbeddedTypstCompiler {  
    fonts: EmbeddedFonts,  
}
```

32. Typst World

Реализовать собственное ограниченное окружение Typst `World`.

Оно должно предоставлять только:

- основной Typst source;
- встроенный шаблон, если он представлен отдельным виртуальным файлом;
- встроенные шрифты;
- заранее разрешённые локальные изображения.

Оно не должно предоставлять:

- общий доступ к файловой системе;
- доступ к домашнему каталогу;
- доступ к текущему каталогу;
- доступ к переменным окружения;
- сетевой доступ;
- загрузку пакетов;
- Typst Universe;
- системные шрифты;
- время;
- произвольные файлы.

33. Виртуальная файловая система

Использовать виртуальные пути:

```
/main.typ
/template.typ
/mdpdf-resources/000001.png
/mdpdf-resources/000002.jpg
```

World должен обслуживать только явно зарегистрированные `FileId`.

Любой неизвестный путь возвращает ошибку доступа.

33.1. Загрузка изображений

До запуска Typst:

1. пройти по `GeneratedTypst.resources`;
2. разрешить реальный путь относительно `base_dir`;
3. canonicalize путь;
4. убедиться, что файл существует;
5. убедиться, что это обычный файл;
6. проверить допустимый размер;
7. прочитать байты;
8. зарегистрировать байты под виртуальным URI.

33.2. Ограничение пути

По умолчанию изображение должно находиться:

- в каталоге исходного Markdown;
- либо в его подкаталогах.

Выход через `..` за корневой каталог запрещён.

Символические ссылки, ведущие за пределы корня, запрещены после canonicalization.

Абсолютные пути в Markdown запрещены.

33.3. Форматы изображений

Первая версия поддерживает:

- PNG;
- JPEG;
- GIF без обязательной анимации;
- SVG без внешних ресурсов.

Неподдерживаемый формат должен выдавать понятную ошибку до вызова Typst.

Определение формата выполняется по содержимому, а не только по расширению.

34. Встроенные шрифты

Минимальный комплект:

- Noto Sans Regular;
- Noto Sans Bold;
- Noto Sans Italic;
- Noto Sans Bold Italic;
- Noto Sans Mono Regular.

Требования:

- байты встраиваются через `include_bytes!`;
- runtime-файлы шрифтов не требуются;
- системные шрифты не используются;
- порядок регистрации шрифтов детерминирован;
- шрифты поддерживают кириллицу;
- лицензии шрифтов включаются в дистрибутив;
- при невозможности разобрать встроенный шрифт программа завершается контролируемой внутренней ошибкой.

Шрифты разбираются один раз на процесс.

Для ленивой инициализации использовать `OnceLock` или эквивалент стандартной библиотеки.

35. Компиляция Typst

Последовательность:

1. создать ограниченный World;
2. зарегистрировать `/main.typ`;
3. зарегистрировать `/template.typ`, если требуется;
4. зарегистрировать изображения;
5. зарегистрировать встроенные шрифты;
6. вызвать Typst compile API;
7. собрать diagnostics;
8. при ошибках прекратить обработку;
9. получить Typst document;
10. передать document в `typst-pdf`;
11. получить PDF bytes;
12. проверить сигнатуру PDF;
13. вернуть `Vec<u8>`.

Фактические вызовы API должны соответствовать закреплённым версиям `typst` и `typst-pdf`.

36. Версии зависимостей

В `Cargo.toml` версии должны быть закреплены совместимо:

```
[dependencies]
pulldown-cmark = "<зафиксированная версия>"
typst = "<одинаковая версия>"
typst-pdf = "<одинаковая версия>"
```

Для `typst` и `typst-pdf` должны использоваться совместимые версии одной линии.

Запрещено:

```
typst = "*"
typst = ">=..."
```

`Cargo.lock` обязателен и хранится в Git.

Обновление Typst выполняется отдельным pull request с:

- changelog review;
- адаптацией `compiler`;
- полным test suite;
- сравнением golden PDF;
- проверкой размера бинарника;

- проверкой лицензий.

37. Диагностика Typst

Внутренние diagnostics Typst преобразуются в собственную модель:

```
pub struct Diagnostic {
  pub severity: Severity,
  pub message: String,
  pub source: Option<DiagnosticSource>,
  pub hints: Vec<String>,
}
```

```
pub enum Severity {
  Warning,
  Error,
}
```

```
pub struct DiagnosticSource {
  pub file: String,
  pub line: Option<usize>,
  pub column: Option<usize>,
}
```

Пользователю нельзя показывать:

- внутренние Rust type names;
- `FileId(...)`;
- debug dump Typst;
- backtrace без `RUST_BACKTRACE`;
- виртуальные детали, не помогающие исправлению.

Пример:

```
input.md:18:1: PDF compilation failed
  image "images/schema.png" could not be decoded
```

Если ошибка возникла в сгенерированном Typst и её нельзя точно сопоставить Markdown, указать:

```
generated Typst:42:17
```

При `--verbose` дополнительно показать связанный Markdown span, если он известен.

38. Предупреждения

Typst warnings не должны теряться.

По умолчанию:

- warnings выводятся в stderr;
- PDF всё равно создаётся.

В будущем допускается `--deny-warnings`, но в первую версию параметр не входит.

39. Проверка PDF

Перед возвратом результата проверить:

- длина не равна нулю;
- начало содержит `%PDF-`;
- PDF имеет разумный минимальный размер;
- exporter не вернул ошибку.

Полный повторный парсинг PDF не требуется.

40. Ограничения ресурсов

Значения первой версии:

- максимальный размер Markdown: 32 MiB;
- максимальное число AST-узлов: 1 000 000;
- максимальная глубина вложенности: 128;
- максимальное число изображений: 1 000;
- максимальный размер одного изображения: 64 MiB;
- максимальный общий размер изображений: 256 MiB;
- максимальная длина URL: 16 KiB;
- максимальная длина одного текстового узла: 16 MiB.

Превышение ограничений возвращает контролируруемую ошибку.

Ограничение времени компиляции внутри процесса первой версии не реализуется, поскольку безопасно остановить зависший вычислительный процесс без отдельного процесса невозможно. Вместо этого запрещается пользовательский Typst-код и внешние шаблоны.

41. Критерии готовности этапа 3

Этап считается завершённым, когда:

- PDF создаётся без установленного Typst;

- не запускаются внешние процессы;
- не используется сеть;
- не используются системные шрифты;
- кириллица корректно попадает в PDF;
- локальные изображения работают;
- попытка прочитать неизвестный файл отклоняется;
- попытка выйти за base directory отклоняется;
- Typst packages недоступны;
- ошибки Typst преобразуются в собственные diagnostics;
- полученный файл начинается с `%PDF-` ;
- end-to-end тесты проходят на целевых платформах;
- бинарник работает в чистом окружении без Typst;
- `cargo clippy` проходит без предупреждений.

Этап 4. CLI и интеграция

42. Pipeline приложения

```
pub fn run(args: Cli) -> Result<ExitStatus, AppError> {
    let source = read_source(&args)?;
    let document = markdown_parser.parse(&source.text)?;
    validate_document(&document)?;

    let generated = typst_generator.generate(&document)?;

    if let Some(path) = args.emit_ast {
        write_ast(path, &document)?;
    }

    if let Some(path) = args.emit_typst {
        write_typst(path, &generated.source)?;
    }

    if args.check {
        compiler.compile(compile_input)?;
        return Ok(ExitStatus::Success);
    }

    let pdf = compiler.compile(compile_input)?;
    write_pdf_atomically(output_path, &pdf, args.overwrite)?;

    Ok(ExitStatus::Success)
}
```

`main.rs` должен быть минимальным:

```
fn main() -> ExitCode {  
    mdpdf::run_cli()  
}
```

43. Коды завершения

```
0  Успех  
1  Общая ошибка выполнения  
2  Ошибка аргументов CLI  
3  Ошибка чтения входа  
4  Ошибка Markdown  
5  Ошибка AST validation  
6  Ошибка генерации Typst  
7  Ошибка компиляции Typst  
8  Ошибка записи результата  
9  Нарушение политики доступа к ресурсу
```

Коды должны быть документированы и стабильны.

44. Вывод

stdout

Используется только для:

- явно запрошенного машинного результата;
- сообщения об успешном создании файла, если не задан `--quiet`.

stderr

Используется для:

- ошибок;
- предупреждений;
- verbose diagnostics.

Сообщение об успехе:

```
Created output.pdf
```

Этап 5. Тестирование

45. Unit-тесты

Обязательные unit-тесты:

- нормализация входа;
 - каждый Markdown AST node;
 - каждый Typst generator node;
 - все escape-функции;
 - URL validation;
 - path validation;
 - resource mapping;
 - diagnostic conversion;
 - output path calculation;
 - atomic output.
-

46. Golden-тесты AST

Для каждого fixture:

```
tests/fixtures/markdown/basic.md
tests/fixtures/expected_ast/basic.json
```

Тест:

1. читает Markdown;
2. строит AST;
3. сериализует нормализованный AST;
4. сравнивает с golden-файлом.

Обновление golden-файлов должно выполняться явной командой, а не автоматически при обычном тесте.

47. Golden-тесты Typst

Для каждого AST/Markdown fixture хранится ожидаемый `.typ`.

Проверяется побайтовое совпадение после нормализации LF.

48. End-to-end тесты

Обязательные сценарии:

1. пустой документ;
2. простой русский текст;
3. все уровни заголовков;
4. жирный, курсив, зачёркивание;
5. inline code;
6. fenced code;
7. вложенные списки;
8. task list;
9. вложенная цитата;
10. таблица;
11. локальная PNG;
12. локальная JPEG;
13. ссылка HTTP;
14. injection payloads;
15. отсутствующее изображение;
16. image path traversal;
17. неверное изображение;
18. очень длинная строка;
19. stdin;
20. существующий output без `--overwrite`.

Проверяется:

- успешный exit code;
- наличие PDF;
- `%PDF -` в начале;
- отсутствие временного файла после завершения;
- отсутствие обращения к внешнему Typst.

49. Визуальные regression-тесты

Минимальный набор эталонных PDF должен создаваться в CI.

Поскольку PDF может содержать меняющиеся служебные данные, побайтовое сравнение PDF не является основным критерием.

Допустимые проверки:

- количество страниц;
- размеры страниц;
- наличие ожидаемых шрифтов;
- наличие ожидаемых изображений;
- рендер первой страницы в тестовом окружении и perceptual comparison.

Инструменты для визуального сравнения могут использоваться только в CI/tests и не являются runtime-зависимостями утилиты.

50. Fuzzing

Создать fuzz targets:

```
fuzz_markdown_parser
fuzz_typst_escape
fuzz_ast_validation
```

Инварианты:

- нет panic;
 - нет выхода за память;
 - парсер всегда завершается;
 - невалидный ввод возвращает `Err`;
 - escape не позволяет создать исполняемый Typst-фрагмент.
-

Этап 6. Качество и поставка

51. Обязательные команды

```
cargo fmt --all --check
cargo check --all-targets --all-features
cargo clippy --all-targets --all-features -- -D warnings
cargo test --all-targets --all-features
cargo doc --no-deps
cargo deny check
```

52. Политика unsafe

В собственном коде проекта:

```
#![forbid(unsafe_code)]
```

Наличие `unsafe` во внешних зависимостях контролируется через dependency review и `cargo deny`, но не может быть полностью запрещено данным атрибутом.

53. Политика ошибок

Использовать:

- `thiserror` в library-модулях;
- единый `AppError` на уровне приложения;
- цепочку источников ошибок;
- человекочитаемое отображение в CLI.

Запрещено:

- `anyhow` внутри доменных модулей;
- ошибки как произвольные строки без типа;
- потеря underlying source;
- `panic!` для пользовательского ввода;
- `unwrap()` и `expect()` в production-коде.

`anyhow` допустим только в тестовых вспомогательных функциях.

54. Логирование

В первой версии отдельный logging framework не обязателен.

Использовать:

- обычные diagnostics;
- `--verbose`;
- `stderr`.

Не добавлять `tracing`, если нет конкретной необходимости.

55. Минимальные зависимости

Предварительный набор:

```
[dependencies]
clap = { version = "...", features = ["derive"] }
pulldown-cmark = "=..."
serde = { version = "...", features = ["derive"] }
serde_json = "..."
thiserror = "..."
typst = "=..."
typst-pdf = "=..."

[dev-dependencies]
assert_cmd = "..."
predicates = "..."
```

```
proptest = "..."  
tempfile = "..."
```

Каждая новая dependency должна иметь обоснование.

Запрещено добавлять dependency ради функции, которая реализуется несколькими очевидными строками стандартной библиотеки.

56. Размер бинарника

Цель release-сборки:

- один бинарный файл;
- отсутствие динамической зависимости от Typst;
- отсутствие внешних font-файлов;
- отсутствие внешнего template-файла.

Размер не является главным показателем лёгкости утилиты, поскольку Typst и шрифты встроены внутрь.

Тем не менее требуется:

- `lto = "thin"` или `"fat"` после измерения;
- `codegen-units = 1`;
- `strip = true`;
- `panic = "abort"` только после подтверждения отсутствия негативного влияния на diagnostics;
- измерение release binary size в CI.

57. Release profile

Начальная конфигурация:

```
[profile.release]  
lto = "thin"  
codegen-units = 1  
strip = true
```

`opt-level = "z"` не применять автоматически: он может ухудшить скорость Typst-компиляции. Решение принимается по benchmark.

58. Производительность

Целевые показатели на обычном современном desktop CPU:

- startup без документа: менее 100 мс;
- Markdown 1 MiB без изображений: полный pipeline менее 2 секунд;
- память для обычного документа до 10 MiB: менее 512 MiB;
- отсутствие повторной загрузки и разбора встроенных шрифтов в рамках одного процесса.

Показатели являются целями, а не жёсткими функциональными требованиями.

59. Документация

README должен содержать:

- назначение;
- установку;
- примеры команд;
- список поддерживаемого Markdown;
- ограничения;
- поведение изображений;
- отсутствие сетевого доступа;
- коды завершения;
- лицензию;
- сведения о встроенных шрифтах.

Документация архитектуры должна объяснять три границы:

```
Markdown parser → AST
AST → Typst generator
Typst source → PDF compiler
```

60. Порядок реализации

Milestone 0 — каркас

Результат:

- Cargo project;
- CLI skeleton;
- error skeleton;
- CI;
- formatting/linting;
- пустые модули;
- встроенный тестовый шрифт;
- зафиксированные dependency versions.

Milestone 1 — Markdown AST

Результат:

- полная модель AST;
- parser;
- validation;
- spans;
- AST golden tests;
- fuzz parser.

Команда проверки:

```
mdpdf input.md --emit-ast ast.json
```

Milestone 2 — Typst generator

Результат:

- встроенный template;
- генерация всех поддерживаемых узлов;
- безопасное escaping;
- resource manifest;
- Typst golden tests;
- injection tests.

Команда проверки:

```
mdpdf input.md --emit-typst output.typ
```

На этом milestone допускается проверять generated Typst установленным Typst только вручную в development, но production-код не должен его вызывать.

Milestone 3 — Embedded Typst compiler

Результат:

- restricted World;
- embedded fonts;
- virtual filesystem;
- local images;
- Typst diagnostics;
- `typst-pdf` ;
- PDF bytes;
- compiler tests.

Milestone 4 — интеграция CLI

Результат:

- полный pipeline;
- stdin;
- output naming;
- atomic writes;
- exit codes;
- check mode;
- overwrite protection.

Milestone 5 — hardening

Результат:

- resource limits;
- path traversal tests;
- fuzzing;
- golden PDFs;
- platform CI;
- dependency audit;
- binary size control;
- release documentation.

61. Definition of Done проекта

Проект считается завершённым, если:

1. команда:

```
mdpdf document.md -o document.pdf
```

создаёт корректный PDF;

1. на машине не установлен Typst;
2. на машине не установлены Chromium, Pandoc и LaTeX;
3. программа не запускает внешние процессы;
4. программа не обращается к сети;
5. программа использует встроенные шрифты;
6. Markdown проходит через собственный AST;
7. Typst generator не зависит от Markdown parser;

8. compiler не зависит от Markdown;
9. поддерживаются заголовки, абзацы, inline formatting, ссылки, изображения, цитаты, списки, task lists, code blocks, таблицы и thematic breaks;
10. кириллица отображается корректно;
11. пользовательский Markdown не может выполнить Typst-код;
12. path traversal через изображения невозможен;
13. все обязательные тесты проходят;
14. `cargo clippy` проходит с `-D warnings`;
15. binary поставляется одним файлом;
16. лицензии встроенных шрифтов и зависимостей соблюдены.